DESIGN AND ANALYSIS OF ALGORITHMS

Pradeesha Ashok, notes by Ramdas Singh

Fifth Semester

Contents

1 Introduction to Algorithms 1
1.1 Asymptotic Notation 1
1.1.1 Properties of Asymptotic Notation 3
1.1.2 Solving Recursions 3
1.2 Divide and Conquer Algorithms 4

Index 7

Chapter 1

Introduction to Algorithms

July 24th.

Project(s) will account for 20% of the final grade. The mid-term exam will account for 30% of the final grade, and the final exam will account for 50%

The design and analysis of algorithms involves two halves, given any algorithm: proving the correctness of the algorithm, and measuring the efficiency of the algorithm. This analysis must be independent of the system's components; henceforth, we will assume the *RAM model of computation* (Random Access Machine model of computation) for our analysis. The RAM model of computation assumes the following:

- every basic operation takes unit (constant) time, and
- memory can be accessed randomly in unit (constant) time.

Consider the simple linear search; the input involves an array A of n elements, and a key. The output is the index of the key in the array, or -1 if the key is not present in the array. The algorithm is as follows:

```
for i = 0 to n-1 do:
    if A[i] == key:
        return i
return -1
```

Here, the best case scenario occurs when the key is present at the first index of the array, and the worst case scenario occurs when the key is not present in the array. Measuring the efficiency of such an algorithm involves looking at how much of the resources (time and space) are consumed, and what the worst case scenario running time is, as a function of the input size.

Definition 1.1. The *input size* is simply defined to be the space needed to store the input.

A possible way of measuring this space is bits; if a number takes, say, 16 bits to store, then the input size of the array above is not n, but rather $\sim 16n$. However, it gets tedious to work with such detailed measurements, and this is where asymptotic analysis comes into play.

1.1 Asymptotic Notation

We first look at *big-*O *notation*; a function f grows at big-O of a function g if such that for large enough inputs, f is bounded by 0 below and a scaling of g above. In other words,

$$\mathrm{O}(g(n)) = \{f(n) \mid \text{there exist } c > 0, n_0 \geq 0 \text{ such that } 0 \leq f(n) \leq c\, g(n) \text{ for all } n \geq n_0\}. \tag{1.1}$$

The notation $f(n) = \mathrm{O}(g(n))$ is an abuse of notation which represents $f(n) \in \mathrm{O}(g(n))$. For examples, we have $10n^2 = \mathrm{O}(n^3)$, $n^2 = \mathrm{O}(n^2)$, $5n^2 + 6n - 8 = \mathrm{O}(n^2)$; in general, one can show that

$$\sum_{i=0}^{d} a_i n^i = \mathrm{O}(n^d) \text{ for } a_d > 0. \tag{1.2}$$

Analogously, we have *big-*Ω *notation*, which is the lower bound of a function:

$$\Omega(g(n)) = \{f(n) \mid \text{there exist } c > 0, n_0 \geq 0 \text{ such that } 0 \leq c\, g(n) \leq f(n) \text{ for all } n \geq n_0\}. \tag{1.3}$$

For examples, $\log n = \Omega(1)$, $2^n = \Omega(n^{1000})$, etc. One may also note that $5n^2 + 6n - 8 = \mathrm{O}(n^2)$; two functions can bound each other via scaling. This leads us to the *big-*Θ *notation* where $f(n) = \Theta(g(n))$ if and only if $f(n) = \mathrm{O}(g(n))$ and $f(n) = \Omega(g(n))$:

$$\Theta(g(n)) = \{f(n) \mid \text{there exist} c_1, c_2 > 0, n_0 \geq 0 \text{ such that } 0 \leq c_1\, g(n) \leq f(n) \leq c_2\, g(n) \text{ for all } n \geq n_0\}. \tag{1.4}$$

Again, one can show $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$. Moreover, functions may be bounded above by a particular function, but not below; this is not an asymptotically tight bound. This leads us to *little-*o *notation*:

$$\mathrm{o}(g(n)) = \{f(n) \mid \text{for all } c > 0, \text{ there exists } n_0 \geq 0 \text{ such that } 0 \leq f(n) < c\, g(n) \text{ for all } n \geq n_0\}. \tag{1.5}$$

Analogously, we have *little-*ω *notation*, to denote a lower bound that is not asymptotically tight:

$$\omega(g(n)) = \{f(n) \mid \text{for all } c > 0, \text{ there exists } n_0 \geq 0 \text{ such that } 0 \leq c\, g(n) < f(n) \text{ for all } n \geq n_0\}. \tag{1.6}$$

Suppose, given two funcitons f and g, the limit

$$L := \lim_{n \to \infty} \frac{f(n)}{g(n)} \tag{1.7}$$

exists. Then, we have the following:

- $L < \infty \implies f(n) = \mathrm{O}(g(n))$,
- $L > 0 \implies f(n) = \Omega(g(n))$.

Coming back to the linear search example, we can easily see that the algorithm is of the order $\mathrm{O}(n)$ and $\Omega(n)$, and hence $\Theta(n)$. We look at the binary search algorithm next; the input involves a sorted array A of n elements, and a key. The output is the index of the key in the array, or -1 if the key is not present in the array. The algorithm is as follows:

```
first = 0; last = n-1
mid = (first+last)//2
while first <= last do:
    if A[mid] == key:
        return mid
    else if A[mid] < key:
        first = mid + 1
    else:
        last = mid - 1
return -1
```

Analyzing this, we see that a single run of the while loop takes $\Theta(1)$ time, and work done outside the loop is $\Theta(1)$ time. So, the time taken by this algorithm is Θ of number of times the while loop is executed; since after every while loop, the effective range is halved, we obtain $\Theta(\log n)$ time for the binary search algorithm. Another classic example is one of the sorting problem; given an input array of n elements, we wish to output it in sorted order. An algorithm for this is as follows:

```
for i = 1 to n-1 do:
    for j = i+1 to n do:
        if A[i] > A[j]:
            swap(A[i], A[j])
return A
```

One may verify that the time taken by this algorithm is $\Theta(n^2)$; this, however, is not efficient enough for large inputs. We will look at more efficient algorithms for sorting later on.

1.1.1 Properties of Asymptotic Notation

Trasitivity

If $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$, then $f(n) = \Theta(h(n))$. This is, in fact, true for all asymptotic notations: we can also show that if $f(n) = \mathrm{O}(g(n))$ and $g(n) = \mathrm{O}(h(n))$, then $f(n) = \mathrm{O}(h(n))$. The same holds for Ω, ω, and o. Let us prove this for Θ notation. Suppose $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$. Then there exist $a_1, b_1 > 0$ and $n_1 \geq 0$ such that

$$0 \leq a_1\, g(n) \leq f(n) \leq b_1\, g(n) \text{ for all } n \geq n_1, \tag{1.8}$$

and there exist $a_2, b_2 > 0$ and $n_2 \geq 0$ such that

$$0 \leq a_2\, h(n) \leq g(n) \leq b_2\, h(n) \text{ for all } n \geq n_2. \tag{1.9}$$

Let $n_0 = \max\{n_1, n_2\}$. Then, for all $n \geq n_0$,

$$0 \leq a_1 a_2\, h(n) \leq a_1\, g(n) \leq f(n) \leq b_1\, g(n) \leq b_1 b_2\, h(n). \tag{1.10}$$

Thus, $f(n) = \Theta(h(n))$.

Reflexivity

We can easily see that $f(n) = \Theta(f(n))$, $f(n) = \mathrm{O}(f(n))$, and $f(n) = \Omega(f(n))$; this is called reflexivity. However, we cannot say that $f(n) = \mathrm{o}(f(n))$ or $f(n) = \omega(f(n))$; this is because the definition of o and ω requires strict inequality. Choosing $c = 1$ in the definition of o and ω leads to a contradiction.

Symmetry

If $f(n) = \Theta(g(n))$, then $g(n) = \Theta(f(n))$. Θ is the only asymptotic notation that is symmetric.

Transpose Symmetry

Simply stated, $f(n) = \mathrm{O}(g(n))$ if and only if $g(n) = \Omega(f(n))$, and $f(n) = \mathrm{o}(g(n))$ if and only if $g(n) = \omega(f(n))$. This is called transpose symmetry.

We state a few statements that hold true in the context of asymptotic notation.

For $a, b > 0$, and $a \neq 1$ and $b \neq 1$,

$$\log_a(n) = \Theta(\log_b(n)). \tag{1.11}$$

Moreover, any polynomial function dominates any logarithmic function; that is, for $k > 0$, and $b > 1$,

$$n^x = \omega(\log_n(n)). \tag{1.12}$$

The exponential function further dominates any polynomial function; that is, for $a > 1$ and $k > 0$,

$$a^n = \omega(n^k). \tag{1.13}$$

As a consequence, this heirarchy of examples holds:

$$\log^2 n < 2^{\sqrt{\log n}} < n^{1/3} < n^2 < n^{2.001} < n^2 \log n < 1.5^n < 2^n = 2^{n+1} < 2^{2n} < n! \tag{1.14}$$

where $<$ denotes that the function on the left is asymptotically dominated by the function on the right.

So far, we have utilized asymptotic notation to analyze the running time of algorithms. We may also use asymptotic notation to analyze the space complexity of algorithms.

1.1.2 Solving Recursions

In many cases, we may not explicitly be given the function $f(n)$, but rather a recursion $T(n)$ the depends on smaller instances of the same problem. For example, in the binary search problem, we have the recursion

$$T(n) = T(n/2) + 1 \tag{1.15}$$

(the 1 simply accounts for unit time; we ultimately require the asymptotic growth). Via induction, one may prove that $T(n) = T(n/2^i) + i$ for any $i \geq 0$. Choosing $i = \log_2 n$, we have $T(n) = T(1) + \log_2 n = \Theta(\log n)$.

Example 1.2. Solve the recursions $T(n) = 2T(n/2) + n$, and $T(n) = T(\sqrt{n}) + 1$.

1.2 Divide and Conquer Algorithms

July 31st.

Algorithms hinged on the *divide and conquer* rule break the problem into smaller subproblems, solve the subproblems recursively, and combine the solutions to the subproblems to solve the original problem. The binary search algorithm is an example of a divide and conquer algorithm. Another example is the merge sort algorithm; given an input array of n elements, we wish to output it in sorted order. The *merge sort* divides the given array into two halves, sorts them recursively, and then merges the sorted halves. The algorithm is as follows:

```
MergeSort(A, n):
    if n = 1:
        return A
    mid = n//2
    left = MergeSort(A[0:mid], mid)
    right = MergeSort(A[mid:n], n-mid)
    return Merge(left, right)

Merge(left, right):
    i = 0; j = 0; k = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            A[k] = left[i]
            i += 1
        else:
            A[k] = right[j]
            j += 1
    # Append any remaining elements from either list
    while i < len(left):
        A[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        A[k] = right[j]
        j += 1
        k += 1
    return A
```

The Merge function combines the two sorted halves into a single sorted array, while the MergeSort function handles the recursive sorting of the two halves. To show the correctness of the algorithm, we can use induction on the size of the array. The base case is when the array has one element, which is trivially sorted. For the inductive step, we assume that MergeSort correctly sorts arrays of size less than n, and we show that it correctly sorts an array of size n by dividing it into two halves, sorting each half recursively, and merging the sorted halves. In the above algorithm, we have `A[i] = min(left[i], right[j])`; this is the key step in the merge process, ensuring that the smallest element from the two halves is placed in the correct position in the merged array.

We compute the time taken; for every iteration of the while loop, an element is added to the merged list. The total time is of the order of size of the merged list, that is, $\mathrm{O}(n)$. Hence, the recursion for the time taken by the merge sort algorithm is $T(n) = 2T(n/2) + \mathrm{O}(n)$.

Example 1.3. Let us solve the recursion $T(n) = aT(n/b) + f(n)$, where $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive. Breaking down recursively, we stop when $n/b^l = \mathrm{O}(1)$, or $l = \log_b n$. Each subsequent leaf then does $a^{\log_b n} = n^{\log_b a}$ work. Thus, the total work done is

$$n^{\log_b a} + f(n) + af(n/b) + a^2 f(n/b^2) + \cdots \tag{1.16}$$

The master method or *master theorem* is exactly for solving recursions of the form $T(n) = a\,T(n/b) + f(n)$, where $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive. The master theorem states that:

1. If $f(n) = \mathrm{O}(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$ and if $af(n/b) \leq cf(n)$ for some $c < 1$ and sufficiently large n, then $T(n) = \Theta(f(n))$.

Thus, using the master theorem, we can solve the recursion for the merge sort algorithm, $T(n) = 2T(n/2) + \mathrm{O}(n)$, where $a = 2$, $b = 2$, and $f(n) = \mathrm{O}(n)$. We have $\log_b a = \log_2 2 = 1$, and $f(n) = \Theta(n^1)$. Hence, we are in case 2 of the master theorem, and we conclude that $T(n) = \Theta(n \log n)$.

Index

big-Ω notation, 2
big-O notation, 1
big-Θ notation, 2

divide and conquer, 4

input size, 1

little-ω notation, 2
little-o notation, 2

master theorem, 5
merge sort, 4

RAM model of computation, 1